Assignment-11.2

Task Description -1 – (Stack Using AI Guidance)

• Task: With the help of AI, design and implement a Stack data
structure supporting basic stack operations.
Expected Output:
• A Python Stack class supporting push, pop, peek, and empty-
check operations with proper documentation.

Prompt

"Create a Python Stack class that supports push, pop, peek, and empty-check operations with proper documentation."

Code:

C: > Users > DELL > Downloads > Untitled-1.py > ...

```python
class Stack:
    """
    Stack Data Structure Implementation
    Stack follows LIFO (Last In First Out) principle.
    """

    def __init__(self):
        # Initialize an empty list to store stack elements
        self.items = []

    def push(self, item):
        """Add an element to the top of the stack"""
        self.items.append(item)

    def pop(self):
        """Remove and return the top element of the stack"""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return the top element without removing it"""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        """Check whether the stack is empty"""
        return len(self.items) == 0


# Example Usage
s = Stack()

s.push(10)
s.push(20)
s.push(30)
```

```python
        return len(self.items) == 0


# Example Usage
s = Stack()

s.push(10)
s.push(20)
s.push(30)

print("Top element:", s.peek())
print("Removed element:", s.pop())
print("Stack empty:", s.is_empty())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.
py
Top element: 30
Removed element: 30
Top element: 30
Removed element: 30
Stack empty: False
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

This prompt clearly instructs the AI to design a **Stack data structure** and include the essential operations.

- **push()** is used to add an element to the top of the stack.
- **pop()** removes the top element following the **LIFO (Last In First Out)** principle.
- **peek()** returns the top element without removing it.
- **is_empty()** checks whether the stack contains elements.
-

Task Description -2 – (Queue Design)

• Task:Use AI assistance to create a Queue data structure

following FIFO principles
Expected Output:
• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

Prompt

"Create a Queue data structure in Python that follows FIFO principle and includes enqueue, dequeue, front element access, and size operations.

```python
class Queue:
    """
    Queue Data Structure Implementation
    Queue follows FIFO (First In First Out) principle
    """

    def __init__(self):
        self.items = []

    def enqueue(self, item):
        # Add element to the queue
        self.items.append(item)

    def dequeue(self):
        # Remove the first element from the queue
        if not self.is_empty():
            return self.items.pop(0)
        return "Queue is empty"

    def front(self):
        # Get the first element
        if not self.is_empty():
            return self.items[0]
        return "Queue is empty"

    def size(self):
        # Return number of elements in queue
        return len(self.items)

    def is_empty(self):
        # Check if queue is empty
        return len(self.items) == 0
```

```python

# Example Usage
q = Queue()

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

print("Front element:", q.front())
print("Removed element:", q.dequeue())
print("Queue size:", q.size())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Removed element: 10
Queue size: 2 …
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\AppData\Local\P
py
Front element: 10
Removed element: 10
Queue size: 2
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

The prompt clearly asks AI to implement a **Queue data structure** using the **FIFO (First In First Out)** principle.

- **enqueue()** adds elements to the queue.
- **dequeue()** removes the first element inserted.
- **front()** returns the first element without removing it.
- **size()** returns the number of elements in the queue.

Task Description -3 –(Singly Linked List Construction)
• Task: Utilize AI to build a singly linked list supporting insertion and traversal.
Expected Output:
• Correctly functioning linked list with node creation, insertion logic, and display functionality.

Prompt

"Create a Singly Linked List in Python that supports node creation, insertion of elements, and traversal (display of elements)

code:

C: > Users > DELL > Downloads > untitled-1.py > ...

```python
class Node:
    # Node class to store data and next reference
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    # Linked List class
    def __init__(self):
        self.head = None

    def insert(self, data):
        # Insert a new node at the end
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    def display(self):
        # Traverse and print elements
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

```

```python
    def display(self):
        # Traverse and print elements
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


# Example Usage
l = LinkedList()
  (method) def insert(data: Any) -> None
l.insert(10)
l.insert(20)
l.insert(30)

l.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.ex
10 -> 20 -> 30 -> None
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

The prompt instructs AI to implement a **Singly Linked List** with essential operations.

- **Node creation** stores data and a reference to the next node.
- **Insertion logic** adds new nodes to the linked list.
- **Traversal (display)** moves through each node to print the elements.

Task Description -4 – (Binary Search Tree Operations)
• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.
Expected Output:
• BST program with correct node insertion and in-order traversal output.

Prompt

"Create a Binary Search Tree (BST) in Python that supports node insertion and in-order traversal."

Code:

```python
class Node:
    # Node class for BST
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    # Insert a node in BST
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        else:
            root.right = self.insert(root.right, data)

        return root

    # In-order traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)


# Example Usage
tree = BST()
root = None
```

```python


# Example Usage
tree = BST()
root = None

root = tree.insert(root, 50)
tree.insert(root, 30)
tree.insert(root, 70)
tree.insert(root, 20)
tree.insert(root, 40)

print("In-order Traversal:")
tree.inorder(root)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\
In-order Traversal:
20 30 40 50 70
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

The prompt instructs AI to implement a **Binary Search Tree (BST)** with insertion and traversal.

- **Insertion** places elements in the correct position based on BST rules.
- If the value is **smaller**, it goes to the **left subtree**.
- If the value is **greater**, it goes to the **right subtree**.

In-order traversal visits nodes in the order:

Task Description -5 – (Hash Table Implementation)
• Task: Create a hash table using AI with collision handling
Expected Output:
• Hash table supporting insert, search, and delete using chaining
or open

Prompt

"Create a Hash Table in Python that supports insert, search, and delete operations with collision handling using chaining."

```python
class HashTable:
    def __init__(self, size):
        # Initialize hash table with empty lists
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        # Simple hash function
        return key % self.size

    def insert(self, key):
        index = self.hash_function(key)
        self.table[index].append(key)

    def search(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            return True
        return False

    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)


# Example Usage
h = HashTable(5)

h.insert(10)
h.insert(15)
h.insert(20)
```

```python
# Example Usage
h = HashTable(5)

h.insert(10)
h.insert(15)
h.insert(20)

print("Search 15:", h.search(15))
h.delete(15)
print("Search 15 after deletion:", h.search(15))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Search 15 after deletion: False
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

The prompt instructs AI to implement a **Hash Table** with basic operations and **collision handling**.

- **insert()** adds elements into the hash table.
- **search()** checks whether an element exists.
- **delete()** removes an element from the table.
- A **hash function** maps keys to indices in the table.